

《嵌入式系统》第四次作业

基于 Android 两次实验项目，参考 Android 指导手册，完成下述任务：

1. 针对第一次 Android 实验中的 53 个程序样例：

a) 界面布局 (5 个)

b) 界面控制 (5 个)

c) 界面事件 (2 个)

d) ActionBar 和菜单 (6 个)

e) 9PatchDemo (1 个)

f) 组件间通讯 (3 个)

g) 数据库系统与访问 (8 个)

h) 多媒体开发 (3 个)

i) 图形图像 (13 个)

j) 网络通信 (2 个)

k) Android 应用项目 (5 个)

从 d)中任选 1 个+f)中任选 1 个+g)中任选一个+i)中任选 1 个+k)中任选 1 个案例，

共计 5 个案例，参考 Android 指导手册第 6 章（程序设计基础中的 6.1.1Android

项目目录结构.和 6.1.2Android 应用解析），进行上述案例的工程文件结构以及核心

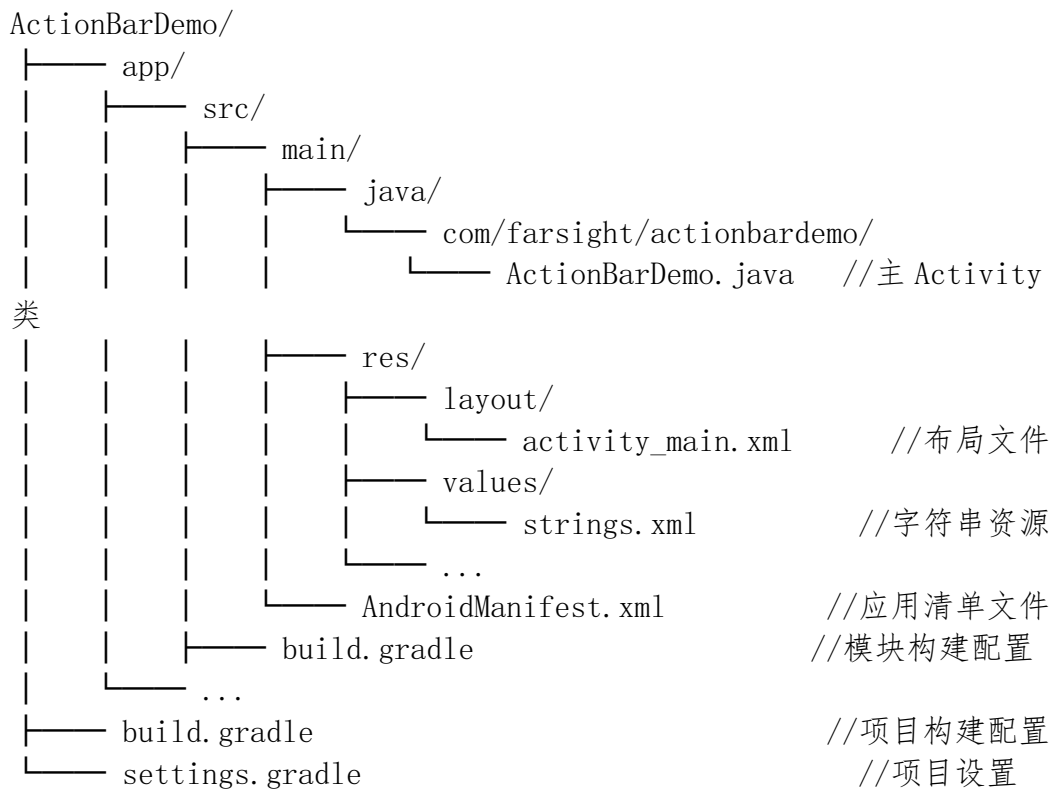
代码中对于不同模块（activity,intent,content provider, service）的实际应用剖

析。

选择：

D: ActionBarDemo

工程结构：



AndroidManifest.xml 结构分析

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <application>
    <!-- Activity组件声明 -->
    <activity android:name=".ActionBarDemo"
      android:theme="@style/Theme.AppCompat.Light"
      android:exported="true">
      <!-- Intent过滤器: 定义应用入口 -->
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
  </application>
</manifest>

```

<activity>: 声明 Activity 组件, 这是 Android 四大组件之一
 android:name=".ActionBarDemo": 指定 Activity 类名
 <intent-filter>: 定义 Intent 过滤器, MAIN/LAUNCHER 表示这是应用入口
 android:exported="true": 允许其他应用启动此 Activity

核心代码剖析:

ActionBarDemo.java

1. `package com.farsight.actionbardemo;`
- 2.

```
3. import androidx.appcompat.app.AppCompatActivity;
4. import androidx.appcompat.app.ActionBar;
5.
6. public class ActionBarDemo extends AppCompatActivity {
7.     private ActionBar actionBar;           // ActionBar 对象引用
8.     private boolean threadOn = false;      // 线程控制标志
9.     private final int HIDE = 1;            // Handler 消息类型: 隐藏
10.    private final int SHOW = 2;             // Handler 消息类型: 显示
11.
12.    // Handler: 用于在主线程中更新 UI
13.    private Handler handler = new Handler() {
14.        @Override
15.        public void handleMessage(Message msg) {
16.            switch (msg.what) {
17.                case HIDE:
18.                    if (actionBar != null) {
19.                        actionBar.hide();      // 隐藏 ActionBar
20.                    }
21.                    break;
22.                case SHOW:
23.                    if (actionBar != null) {
24.                        actionBar.show();      // 显示 ActionBar
25.                    }
26.                    break;
27.            }
28.        }
29.    };
30.
31.    @Override
32.    protected void onCreate(Bundle savedInstanceState) {
33.        super.onCreate(savedInstanceState);
34.        setContentView(R.layout.activity_main);
35.
36.        // 获取 ActionBar 实例 (使用 Support 库)
37.        actionBar = getSupportActionBar();
38.
39.        if (actionBar != null) {
40.            actionBar.setTitle("华清远见");    // 设置 ActionBar 标题
41.            actionBar.hide();                  // 初始状态: 隐藏
42.        }
43.
44.        // 启动后台线程, 循环显示/隐藏 ActionBar
45.        actionBarHideAndShow = new ActionBarHideAndShow();
46.        threadOn = true;
```

```

47.     actionBarHideAndShow.start();
48. }
49.
50. // 内部线程类: 实现 ActionBar 的自动显示/隐藏
51. private class ActionBarHideAndShow extends Thread {
52.     @Override
53.     public void run() {
54.         while (threadOn) {
55.             try {
56.                 sleep(500); // 每 500ms 执行一次
57.             } catch (InterruptedException e) {
58.                 e.printStackTrace();
59.             }
60.             count--;
61.             if (count == 0) {
62.                 count = 10;
63.                 // 切换显示/隐藏状态
64.                 old = old == HIDE ? SHOW : HIDE;
65.                 // 通过 Handler 发送消息到主线程
66.                 handler.sendMessage(old);
67.             }
68.         }
69.     }
70. }
71. }

```

1. Activity 组件应用:

extends AppCompatActivity: 继承自支持库的 Activity 基类

onCreate(): Activity 生命周期方法, 初始化 UI 和逻辑

2. Handler 机制:

Android 不允许在子线程中直接更新 UI

Handler 用于实现线程间通信, 将子线程的消息传递到主线程

handleMessage(): 处理来自子线程的消息

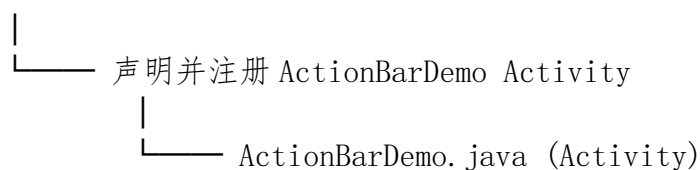
3. ActionBar 操作:

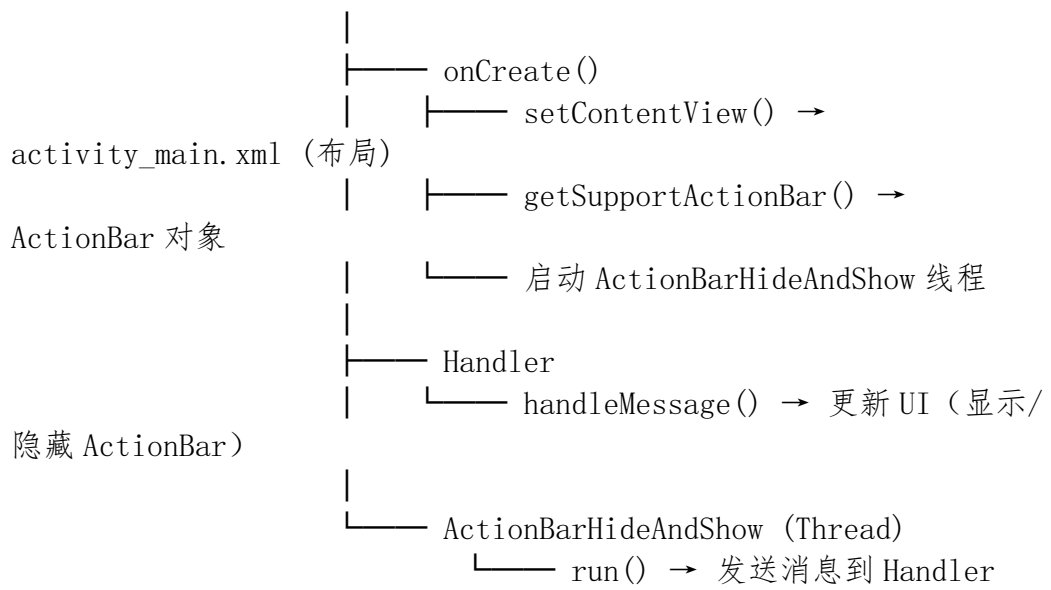
getSupportActionBar(): 获取 ActionBar 实例

actionBar.hide()/show(): 控制 ActionBar 的显示和隐藏

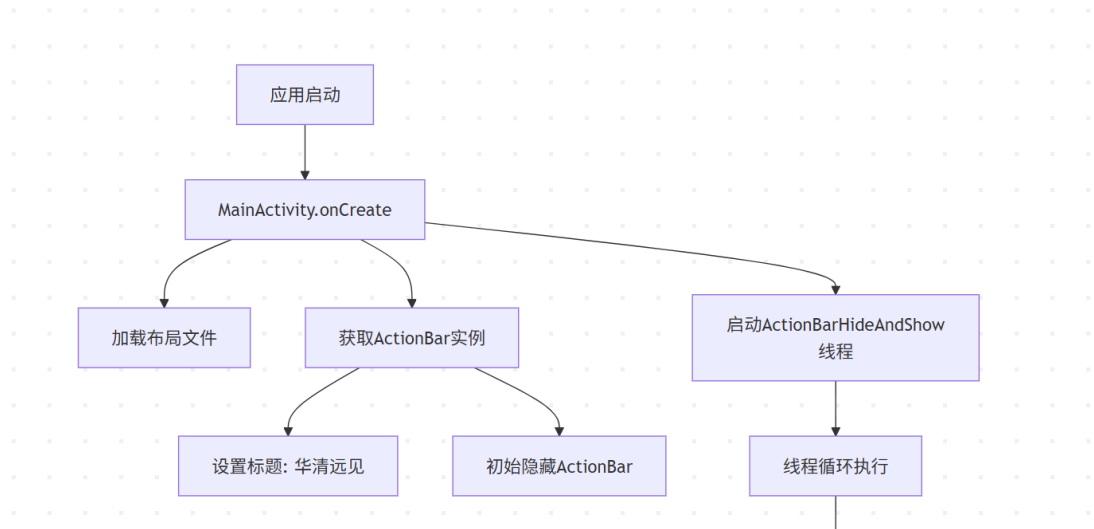
文件调用关系图:

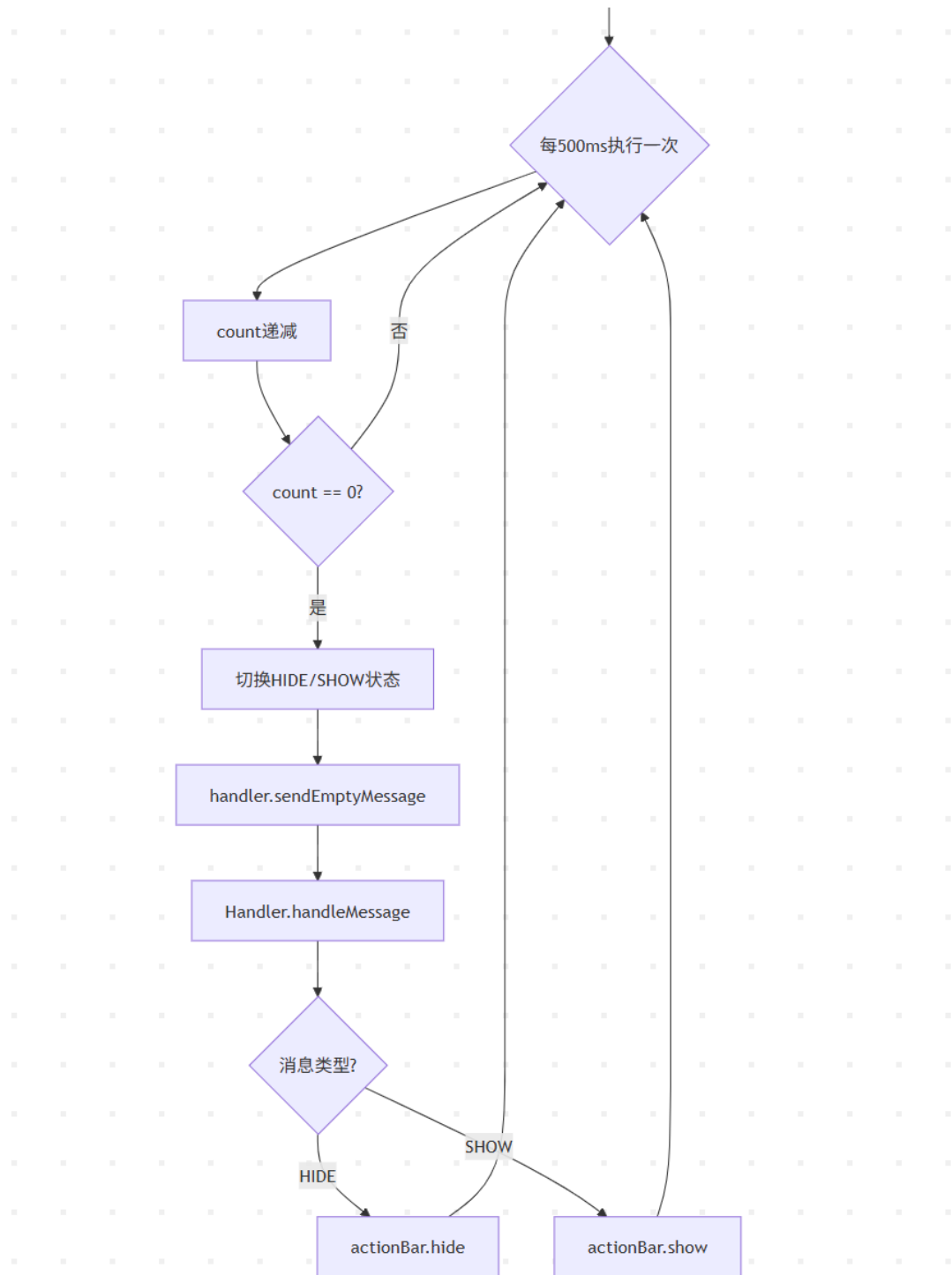
AndroidManifest.xml





流程图：





F: ActivityCommunication

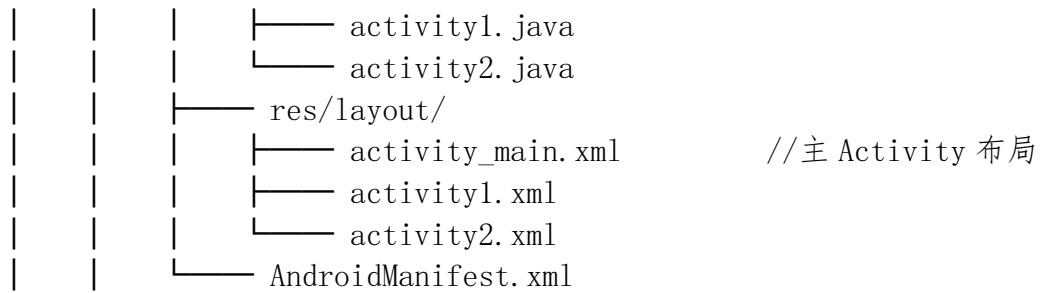
工程结构:

ActivityCommunication/

```

├── app/
│   ├── src/main/
│   │   ├── java/com/farsight/activitycommunication/
│   │   └── MainActivity.java           //主 Activity

```



AndroidManifest.xml 分析

```
<application>
    <!-- 主Activity -->
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <!-- 子Activity1: 用于数据输入和返回 -->
    <activity android:name=".activity1" />

    <!-- 子Activity2: 用于简单返回 -->
    <activity android:name=".activity2" />
</application>
```

三个 Activity 都在清单文件中注册
只有 MainActivity 有 LAUNCHER intent-filter，作为应用入口
activity1 和 activity2 没有 intent-filter，只能通过显式 Intent 启动

核心代码:

MainActivity.java

```

1. public class MainActivity extends AppCompatActivity {
2.     private static final int SUBACTIVITY1 = 1;        // Activity1 请求码
3.     private static final int SUBACTIVITY2 = 2;        // Activity2 请求码
4.     TextView textView;
5.
6.     @Override
7.     protected void onCreate(Bundle savedInstanceState) {
8.         super.onCreate(savedInstanceState);
9.         setContentView(R.layout.activity_main);
10.
11.         textView = (TextView) findViewById(R.id.textshow);
12.         Button button1 = (Button) findViewById(R.id.button1);
13.         Button button2 = (Button) findViewById(R.id.button2);
14.
15.         // 按钮 1: 启动 Activity1 并等待返回结果

```

```

16.     button1.setOnClickListener(new View.OnClickListener() {
17.         @Override
18.         public void onClick(View view) {
19.             // 创建显式 Intent, 指定目标 Activity
20.             Intent intent = new Intent(MainActivity.this, activity1.class);
21.             // 启动 Activity 并等待结果 (使用 startActivityForResult)
22.             startActivityForResult(intent, SUBACTIVITY1);
23.         }
24.     });
25.
26.     // 按钮 2: 启动 Activity2 并等待返回结果
27.     button2.setOnClickListener(new View.OnClickListener() {
28.         @Override
29.         public void onClick(View view) {
30.             Intent intent = new Intent(MainActivity.this, activity2.class);
31.             startActivityForResult(intent, SUBACTIVITY2);
32.         }
33.     });
34. }
35.
36. // 接收子 Activity 返回的结果
37. @Override
38. protected void onActivityResult(int requestCode, int resultCode, Intent data) {
39.     super.onActivityResult(requestCode, resultCode, data);
40.
41.     switch(requestCode){
42.         case SUBACTIVITY1:
43.             // 检查返回结果是否成功
44.             if (resultCode == RESULT_OK){
45.                 // 从 Intent 中获取返回的数据 (URI)
46.                 Uri uriData = data.getData();
47.                 textView.setText(uriData.toString());
48.             }
49.             break;
50.         case SUBACTIVITY2:
51.             // Activity2 没有返回数据
52.             break;
53.     }
54. }
55. }

```

Intent 机制解析:

1. 显式 Intent:

`new Intent(MainActivity.this, activity1.class)`: 明确指定目标

Activity

用于应用内部组件间通信

2. startActivityForResult():

启动 Activity 并等待返回结果

第二个参数是请求码 (requestCode)，用于区分不同的请求

3. onActivityResult():

接收子 Activity 返回的结果

requestCode: 区分是哪个 Activity 返回的

resultCode: 返回状态 (RESULT_OK/RESULT_CANCELED)

data: 返回的数据 (Intent 对象)

activity1.java: 返回数据

```
public class activity1 extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity1);

        final EditText editText = (EditText)findViewById(R.id.text1);
        Button btnOK = (Button)findViewById(R.id.button1);
        Button btnCancel = (Button)findViewById(R.id.button2);

        // OK按钮: 返回数据
        btnOK.setOnClickListener(new View.OnClickListener(){
            public void onClick(View view){
                String uriString = editText.getText().toString();
                Uri data = Uri.parse(uriString); // 将字符串转换为URI
                Intent result = new Intent(null, data); // 创建结果Intent
                setResult(RESULT_OK, result); // 设置返回结果
                finish(); // 关闭Activity
            }
        });

        // Cancel按钮: 取消操作
        btnCancel.setOnClickListener(new View.OnClickListener(){
            public void onClick(View view){
                setResult(RESULT_CANCELED, null); // 设置取消状态
                finish();
            }
        });
    }
}
```

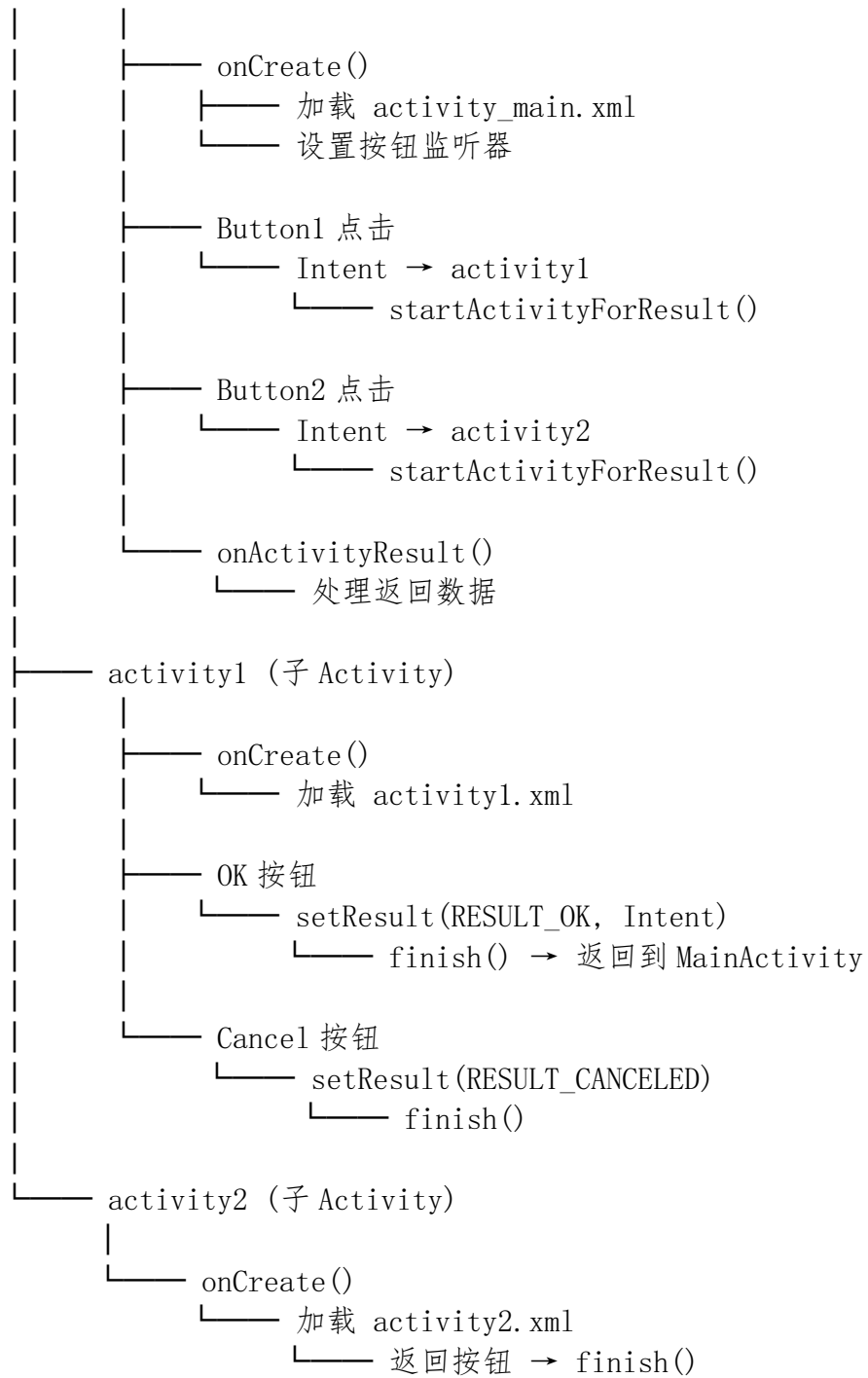
setResult(): 设置返回给父 Activity 的结果

finish(): 关闭当前 Activity，返回到父 Activity

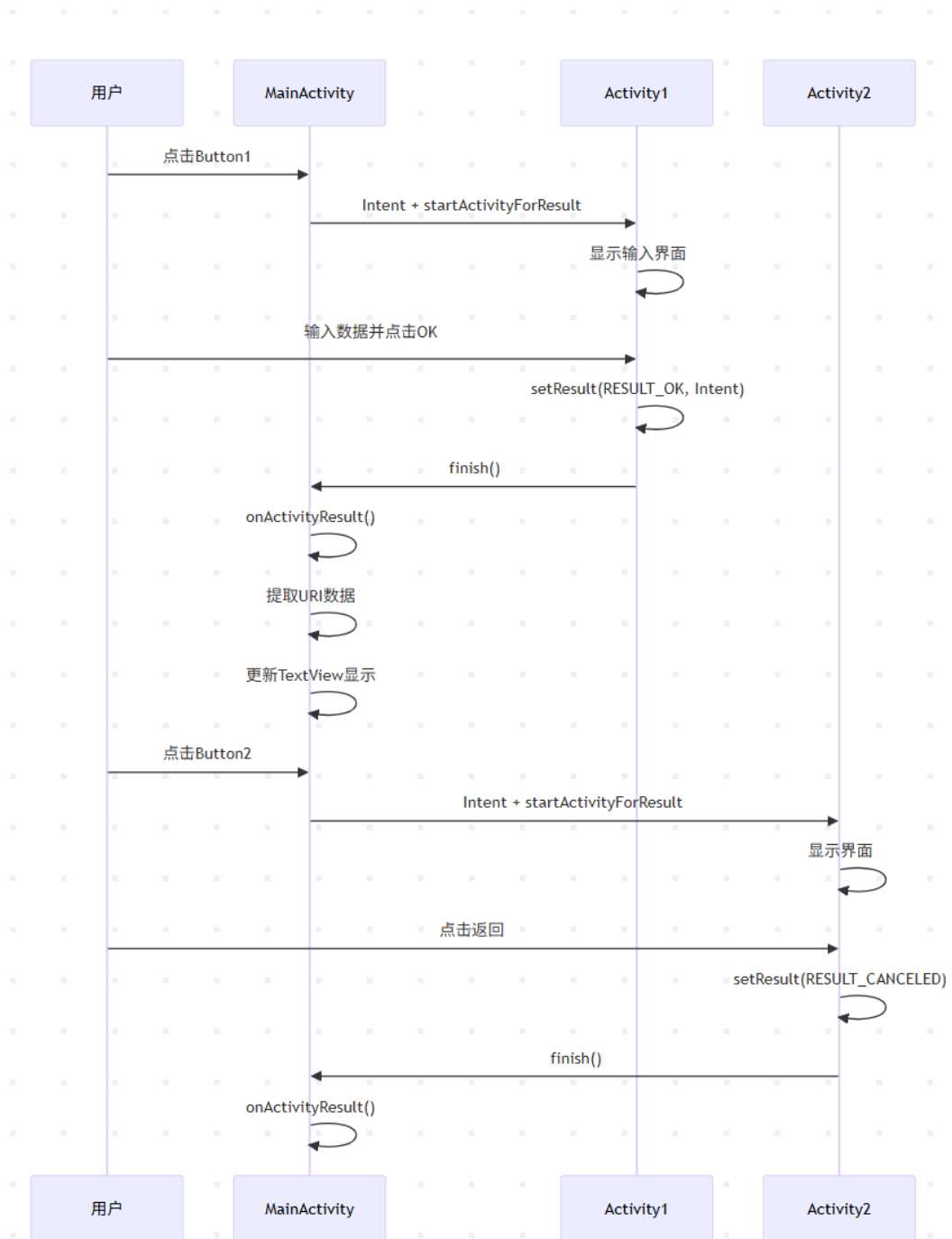
文件调用关系图

AndroidManifest.xml





Activity 间通信流程图



G: SQLiteExam2

工程结构:

SQLiteExam2/

|—— app/src/main/

| |—— java/com/farsight/sqliteexam2/

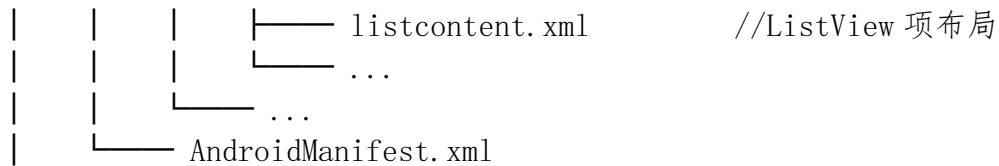
| | |—— MainActivity.java

//主 Activity (包含

数据库操作)

| |—— res/

| | |—— layout/



核心代码

MainActivity.java

```
1. public class MainActivity extends AppCompatActivity {
2.     private final String DATABASE_NAME = "school";           // 数据库名称
3.     private SQLiteDatabase db;                               // 数据库对象
4.     private ListView lv;                                     // 列表视图
5.     private ArrayList<Map<String, Object>> data;             // 数据列表
6.
7.     @Override
8.     public void onCreate(Bundle savedInstanceState) {
9.         super.onCreate(savedInstanceState);
10.
11.         // 创建 ListView (动态创建, 未使用 XML 布局)
12.         lv = new ListView(this);
13.
14.         // 创建数据库辅助类实例
15.         MyDatabaseHelper myDBHelper = new MyDatabaseHelper(
16.             this, DATABASE_NAME, null, 3);
17.
18.         // 获取可写数据库 (如果不存在会自动创建)
19.         db = myDBHelper.getWritableDatabase();
20.
21.         // 初始化数据: 插入一条学生记录
22.         initData("tom", "09393", "1106");
23.
24.         // 查询 STUDENTINFO 表中的所有数据
25.         String sql = "SELECT * FROM STUDENTINFO";
26.         Cursor result = db.rawQuery(sql, null);
27.
28.         // 将查询结果转换为 List
29.         data.clear();
30.         while (!result.isLast()) {
31.             result.moveToNext();
32.             HashMap<String, Object> mymap = new HashMap<>();
33.             // 遍历所有列
34.             for (int i = 0; i < result.getColumnCount(); i++) {
35.                 mymap.put(result洗getColumnName(i), result.getString(i));
36.             }
37.             data.add(mymap);
```

```

38.     }
39.
40.     // 创建 SimpleAdapter 适配器
41.     SimpleAdapter adapter = new SimpleAdapter(
42.         this, data,
43.         R.layout.listcontent,           // 列表项布局
44.         new String[] { "ID", "NAME", "PHONE", "CLASS" }, // 数据键
45.         new int[] { R.id.textView1, R.id.textView2,      // 视图 ID
46.             R.id.textView3, R.id.textView4 });
47.
48.     lv.setAdapter(adapter);
49.     setContentView(lv);
50.
51.     result.close();                    // 关闭 Cursor
52.     db.close();                       // 关闭数据库
53. }
54.
55. // 插入数据到数据库
56. private void initData(String name, String phone, String sclass) {
57.     ContentValues values = new ContentValues(); // 键值对容器
58.     values.put("NAME", name);
59.     values.put("PHONE", phone);
60.     values.put("CLASS", sclass);
61.     // 插入数据 (ID 为自增主键, 自动生成)
62.     db.insert("STUDENTINFO", "ID", values);
63. }
64.
65. // 数据库辅助类: 继承 SQLiteOpenHelper
66. class MyDatabaseHelper extends SQLiteOpenHelper {
67.     public String DATABASE_TABLE = "STUDENTINFO";
68.
69.     // 创建表的 SQL 语句
70.     public final String DB_CREATE_TABLE =
71.         "CREATE TABLE " + DATABASE_TABLE
72.         + "( " + "ID    INTEGER    NOT NULL,"
73.         + "NAME  CHAR(20)  NOT NULL,"
74.         + "PHONE CHAR(20), "
75.         + "CLASS CHAR(50), "
76.         + "PRIMARY KEY(ID) );";
77.
78.     // 构造函数
79.     public MyDatabaseHelper(Context context, String name,
80.         CursorFactory factory, int version) {
81.         super(context, name, factory, version);

```

```

82.     }
83.
84.     // 数据库首次创建时调用
85.     @Override
86.     public void onCreate(SQLiteDatabase db) {
87.         db.execSQL(DB_CREATE_TABLE);           // 执行建表 SQL
88.     }
89.
90.     // 数据库版本升级时调用
91.     @Override
92.     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
93.         // 升级逻辑 (本案例未实现)
94.     }
95. }
96. }

```

SQLite 数据库关键点:

1. SQLiteOpenHelper:

管理数据库的创建和版本控制

onCreate(): 首次创建数据库时调用

onUpgrade(): 数据库版本升级时调用

2. 数据库操作:

getWritableDatabase(): 获取可写数据库

db.insert(): 插入数据

db.rawQuery(): 执行 SQL 查询

Cursor: 查询结果游标, 用于遍历数据

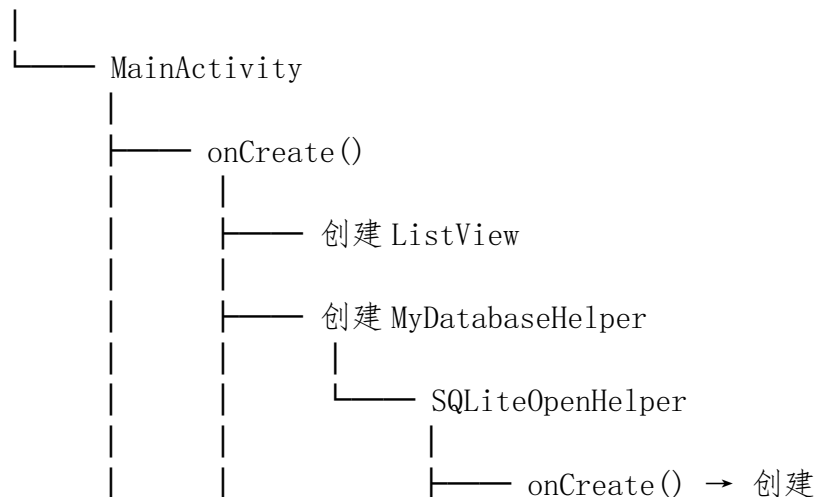
3. ContentValues:

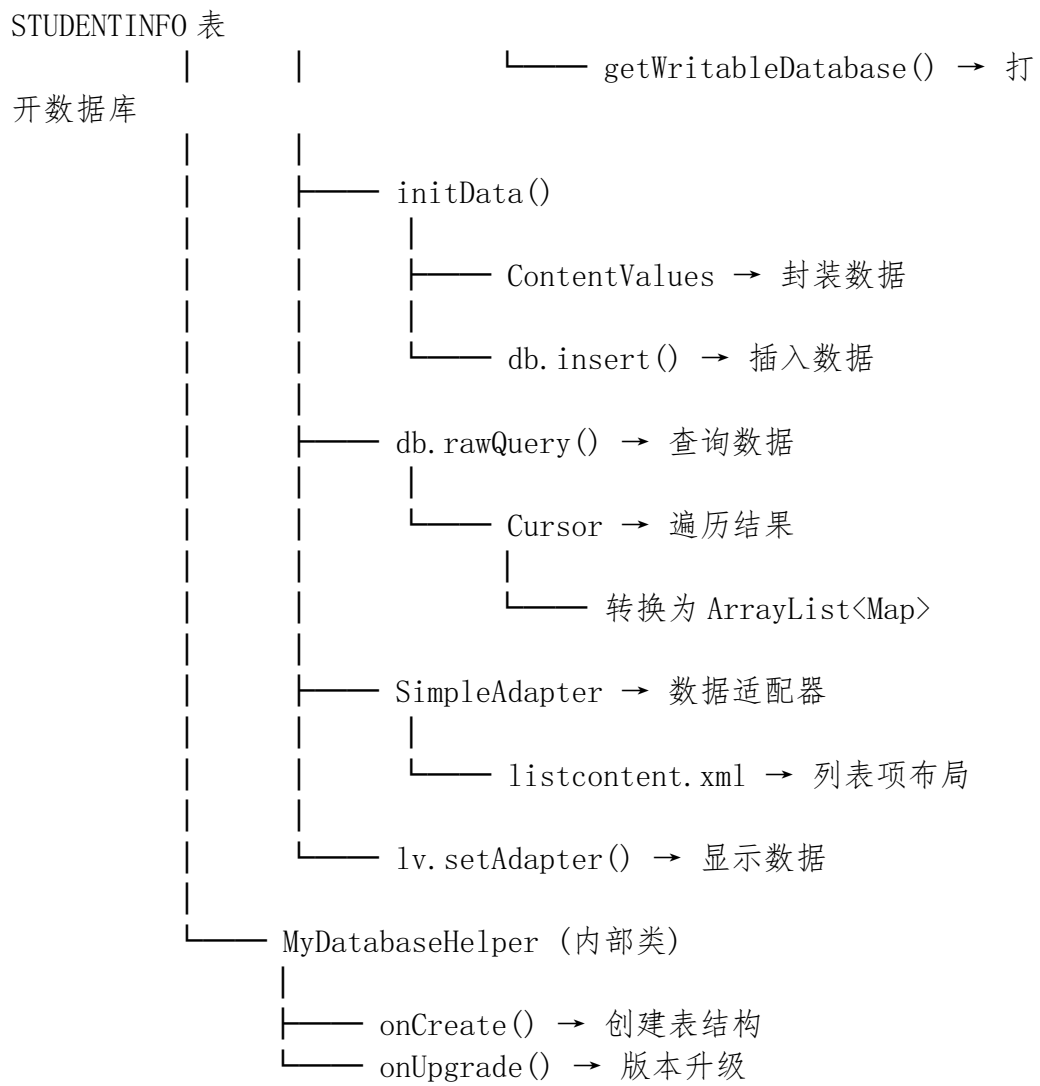
用于存储键值对数据

配合 insert() 和 update() 使用

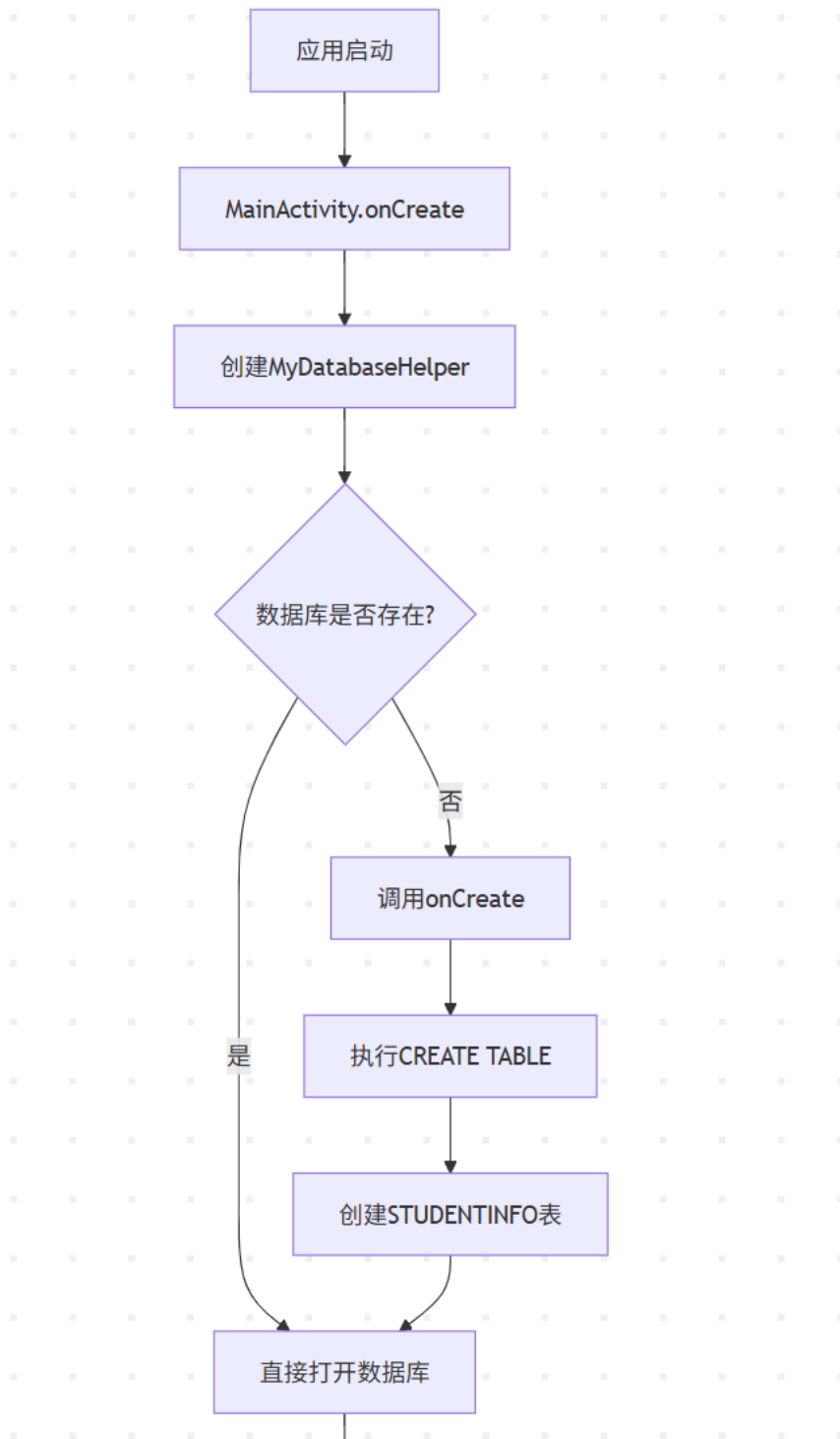
文件调用关系图

AndroidManifest.xml





数据库操作流程图



I: GraphicAnimation

工程结构:

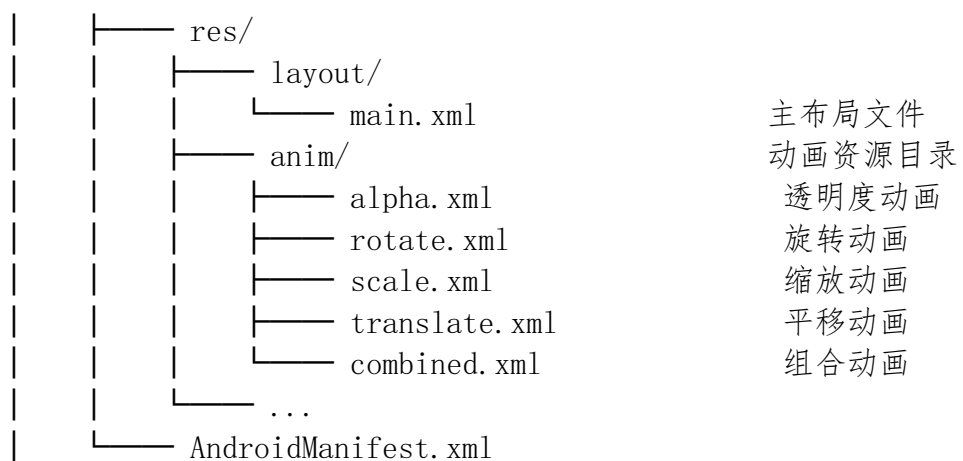
GraphicAnimation/

|—— app/src/main/

| |—— java/com/farsight/graphicanimation/

| |—— MainActivity.java

主 Activity



核心代码

```

// 通用XML动画加载方法
private void xmlAnimation(int resourceId) {
    Animation animation = AnimationUtils.loadAnimation(this, resourceId);
    mImageView.startAnimation(animation);
}

// 代码方式: 创建缩放动画
private void scaleStatic() {
    Animation scaleAnimation = new ScaleAnimation(
        0f, 1f, // x轴起始和结束缩放比例
        0f, 1f, // y轴起始和结束缩放比例
        Animation.RELATIVE_TO_SELF, 0.5f, // x轴缩放中心点
        Animation.RELATIVE_TO_SELF, 0.5f); // y轴缩放中心点
    scaleAnimation.setDuration(3000); // 动画持续时间3秒
    mImageView.startAnimation(scaleAnimation);
}

// 组合动画: 使用AnimationSet
private void combinedSetStatic() {
    AnimationSet animationSet = new AnimationSet(true);

    // 从XML加载各个动画
    Animation scaleAnimation = AnimationUtils.loadAnimation(this,
anim.scale);
    Animation rotateAnimation = AnimationUtils.loadAnimation(this,
anim.rotate);
    Animation translateAnimation = AnimationUtils.loadAnimation(this,
anim.translate);
    Animation alphaAnimation = AnimationUtils.loadAnimation(this,
anim.alpha);

    // 添加到动画集合
    animationSet.addAnimation(rotateAnimation);
    animationSet.addAnimation(translateAnimation);
    animationSet.addAnimation(alphaAnimation);
    animationSet.addAnimation(scaleAnimation);

    // 设置插值器
    animationSet.setInterpolator(this, android.R.anim.linear_interpolator);

    // 启动组合动画
    mImageView.startAnimation(animationSet);
}

```

动画机制解析:

1. Animation 类体系:

AlphaAnimation: 透明度动画

RotateAnimation: 旋转动画

ScaleAnimation: 缩放动画

TranslateAnimation: 平移动画

AnimationSet: 动画集合, 可组合多个动画

2. 两种创建方式:

代码方式: 使用 Java 代码创建 Animation 对象

XML 方式: 在 res/anim/目录下定义 XML, 使用
AnimationUtils.loadAnimation() 加载

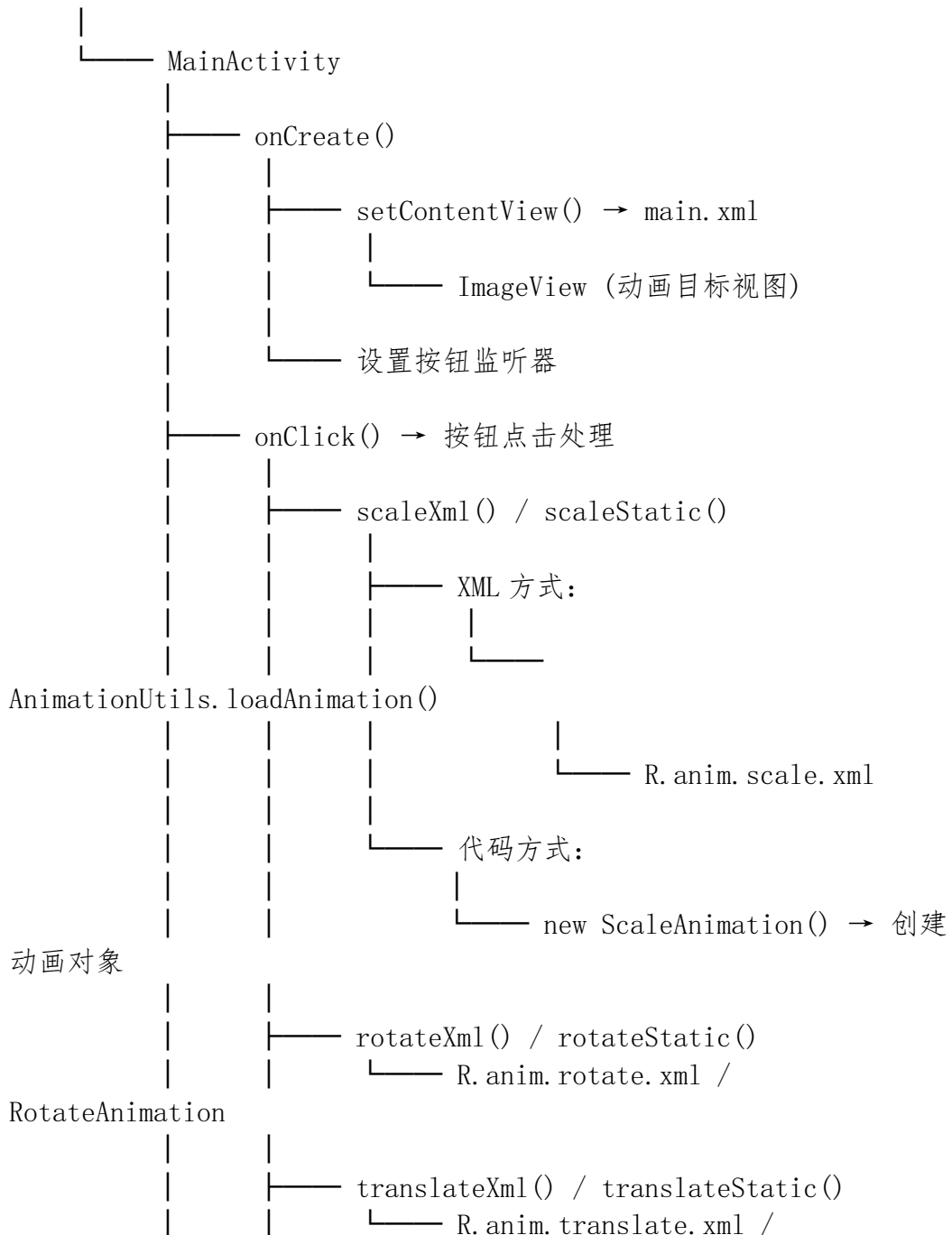
3. AnimationUtils:

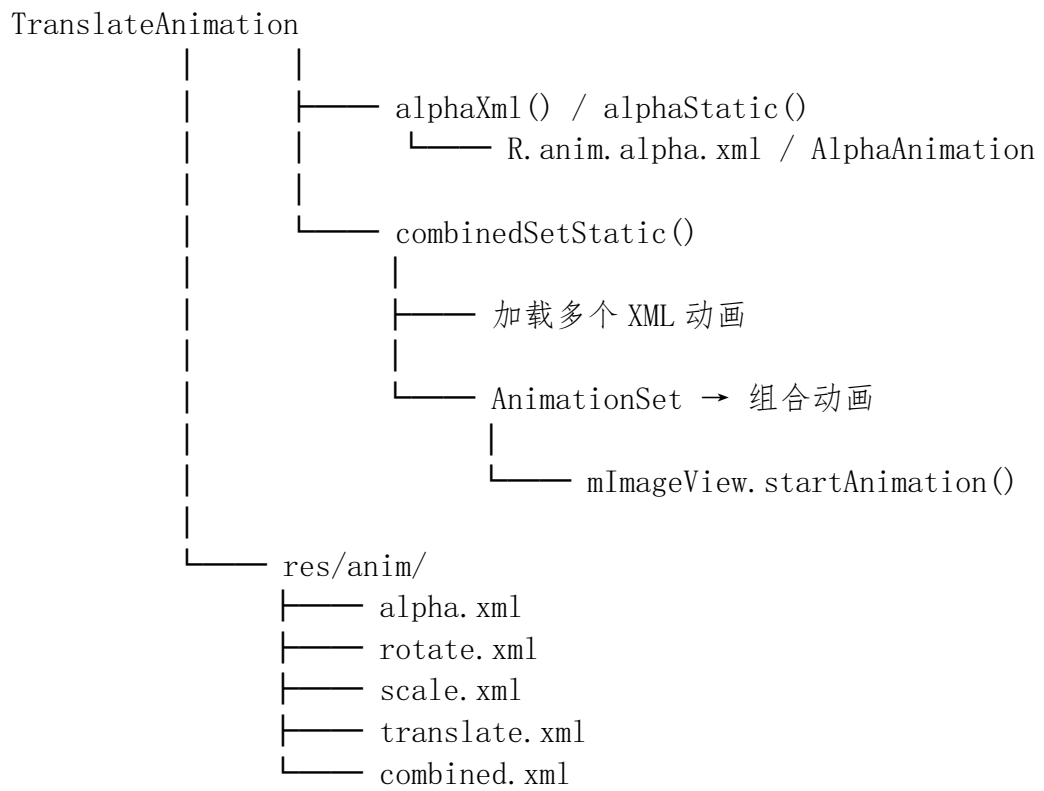
loadAnimation(): 从 XML 资源加载动画

简化动画资源的加载过程

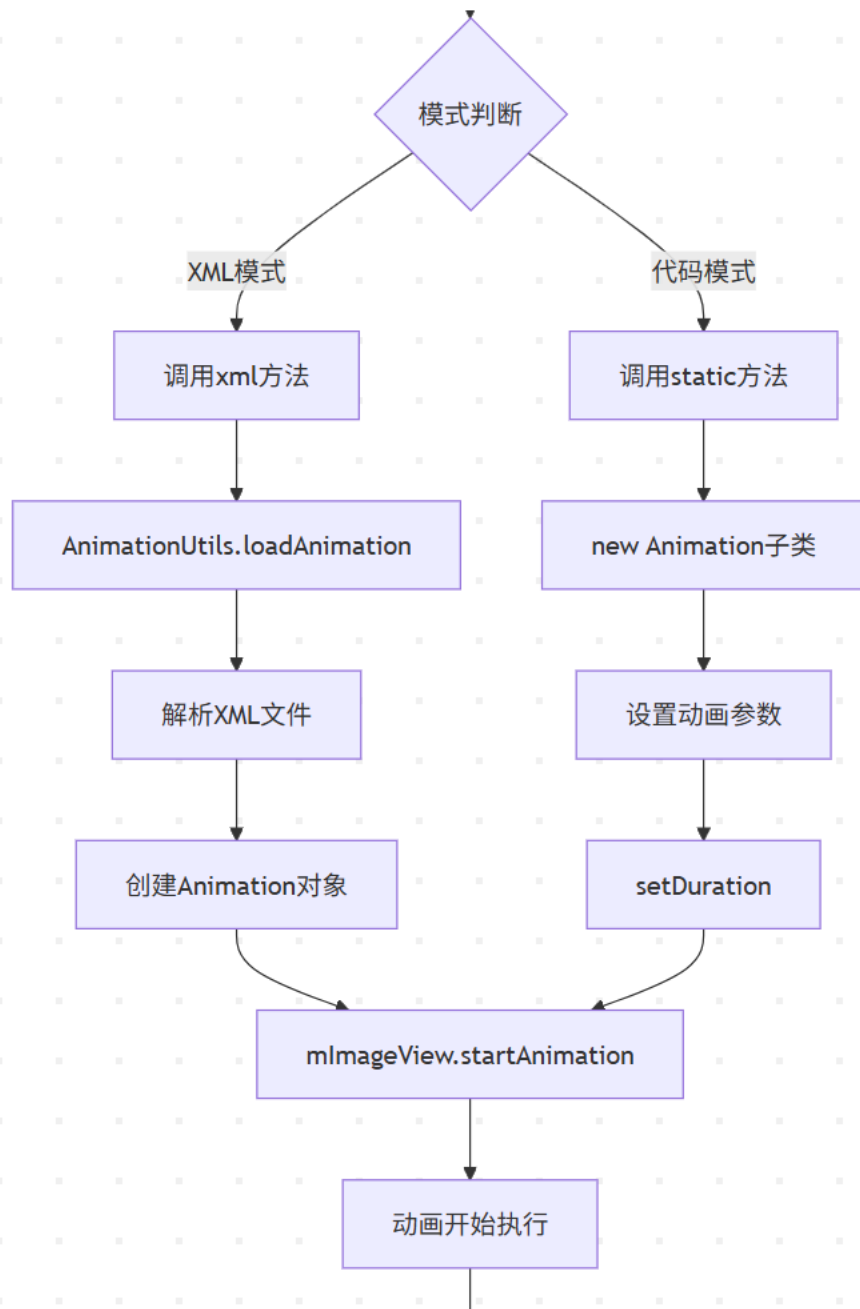
文件调用关系图:

AndroidManifest.xml





流程图



K: NoteApp

工程结构:

NoteApp/

|—— app/src/main/

| |—— java/com/farsight/noteapp/

| |—— MainActivity.java

主 Activity (笔

记列表)

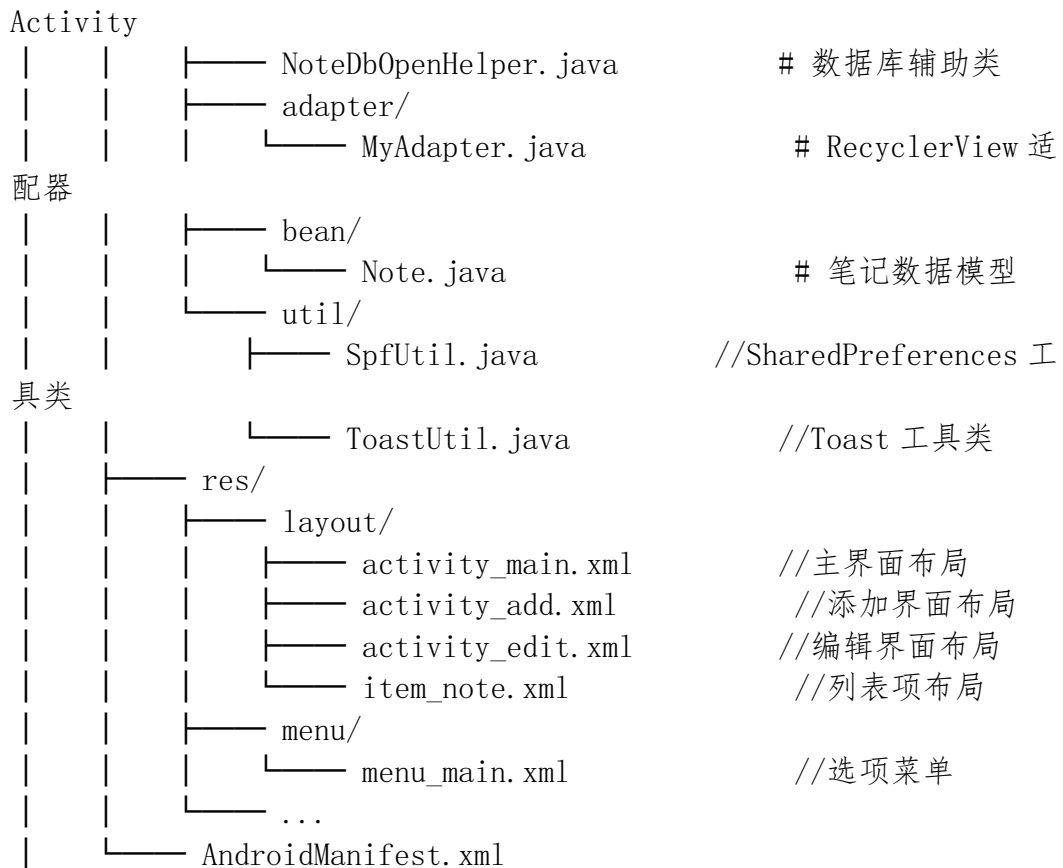
| |—— AddActivity.java

添加笔记

Activity

| |—— EditActivity.java

编辑笔记



AndroidManifest.xml 分析

```

<application>
  <!-- 主Activity: 笔记列表 -->
  <activity android:name=".MainActivity"
    android:label="记事列表"
    android:exported="true">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>

  <!-- 添加笔记Activity -->
  <activity android:name=".AddActivity"
    android:label="添加记事"
    android:parentActivityName=".MainActivity" />

  <!-- 编辑笔记Activity -->
  <activity android:name=".EditActivity"
    android:label="修改记事"
    android:parentActivityName=".MainActivity" />
</application>
  
```

三个 Activity 组成完整的应用

android:parentActivityName: 定义父 Activity, 用于导航返回
使用 Intent 在 Activity 间传递数据

核心代码：

```
@Override
protected void onResume() {
    super.onResume();
    // Activity恢复时刷新数据
    refreshDataFromDb();
    setListLayout();
}

private void initData() {
    mNotes = new ArrayList<>();
    mNoteDbOpenHelper = new NoteDbOpenHelper(this);
}

// 从数据库获取所有笔记
private List<Note> getDataFromDB() {
    return mNoteDbOpenHelper.queryAllFromDb();
}

// 刷新数据
private void refreshDataFromDb() {
    mNotes = getDataFromDB();
    mAdapter.refreshData(mNotes);
}

// 添加按钮点击: 启动AddActivity
public void add(View view) {
    Intent intent = new Intent(this, AddActivity.class);
    startActivity(intent);           // 启动Activity
}

// 创建选项菜单
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);

    // 搜索功能
    SearchView searchView = (SearchView) menu.findItem(R.id.menu_search)
        .getActionView();

    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener()

        @Override
        public boolean onQueryTextChange(String newText) {
            // 根据标题搜索笔记
            mNotes = mNoteDbOpenHelper.queryFromDbByTitle(newText);
            mAdapter.refreshData(mNotes);
            return true;
        }
    });
}
```

关键组件应用：

1. Activity 生命周期：

onCreate(): 初始化

onResume(): 每次显示时刷新数据

2. Intent 应用:

new Intent(this, AddActivity.class): 显式 Intent 启动 AddActivity

startActivity(intent): 启动 Activity

3. SQLite 数据库:

NoteDbOpenHelper: 数据库辅助类

queryAllFromDb(): 查询所有笔记

queryFromDbByTitle(): 按标题搜索

NoteDbOpenHelper.java

```
// 创建表的SQL语句
private static final String CREATE_TABLE_SQL =
    "create table " + TABLE_NAME_NOTE
    + " (id integer primary key autoincrement, "
    + "title text, content text, create_time text)";

public NoteDbOpenHelper(Context context) {
    super(context, DB_NAME, null, 1);
}

@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(CREATE_TABLE_SQL); // 创建表
}

// 插入数据
public long insertData(Note note) {
    SQLiteDatabase db = getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("title", note.getTitle());
    values.put("content", note.getContent());
    values.put("create_time", note.getCreatedTime());
    return db.insert(TABLE_NAME_NOTE, null, values);
}

// 查询所有数据
public List<Note> queryAllFromDb() {
    SQLiteDatabase db = getWritableDatabase();
    List<Note> noteList = new ArrayList<>();

    Cursor cursor = db.query(TABLE_NAME_NOTE, null, null, null, null, null, null);
    if (cursor != null) {
        while (cursor.moveToNext()) {
            Note note = new Note();
            note.setId(cursor.getString(cursor.getColumnIndex("id")));
        }
    }
}
```

AddActivity.java: internet 返回


```

public class AddActivity extends AppCompatActivity {
    private EditText etTitle, etContent;
    private NoteDbOpenHelper mNoteDbOpenHelper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_add);
        mNoteDbOpenHelper = new NoteDbOpenHelper(this);
    }

    // 添加笔记
    public void add(View view) {
        String title = etTitle.getText().toString();
        String content = etContent.getText().toString();

        if (TextUtils.isEmpty(title)) {
            ToastUtil.toastShort(this, "标题不能为空!");
            return;
        }

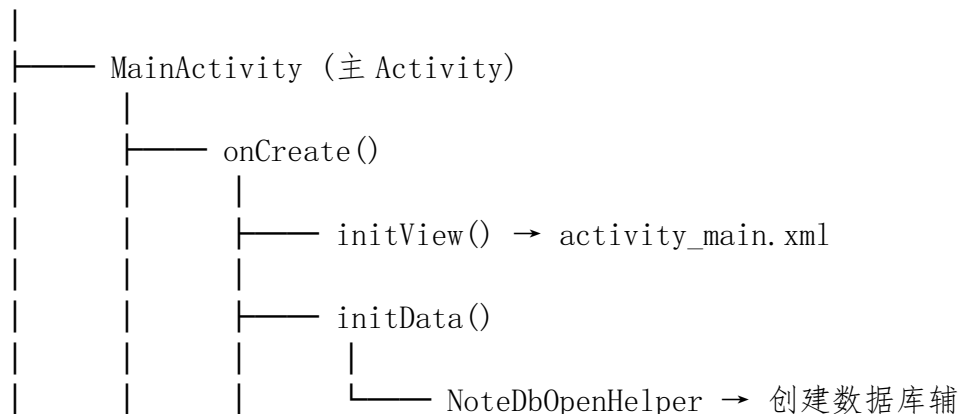
        Note note = new Note();
        note.setTitle(title);
        note.setContent(content);
        note.setCreateTime(getCurrentTimeFormat());

        // 插入数据库
        long row = mNoteDbOpenHelper.insertData(note);
        if (row != -1) {
            ToastUtil.toastShort(this, "添加成功!");
            this.finish(); // 关闭Activity, 返回
        }
    }
}

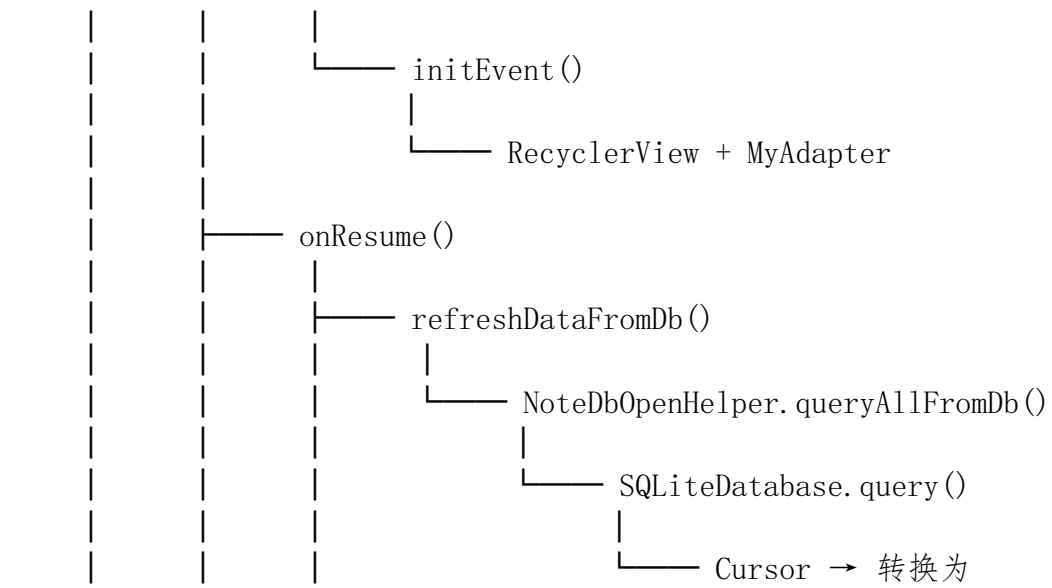
```

文件调用关系图：

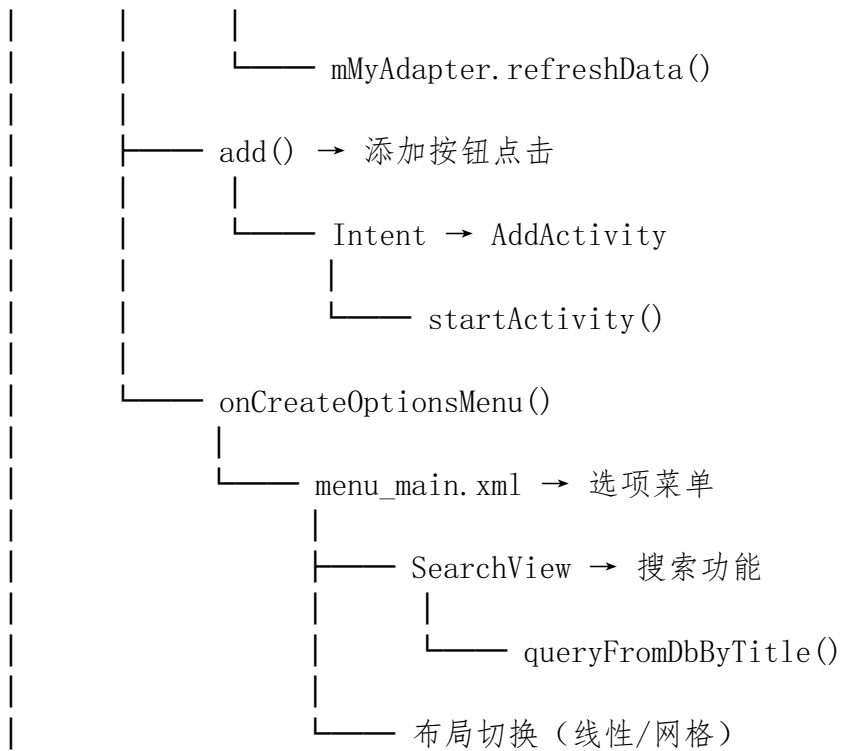
AndroidManifest.xml



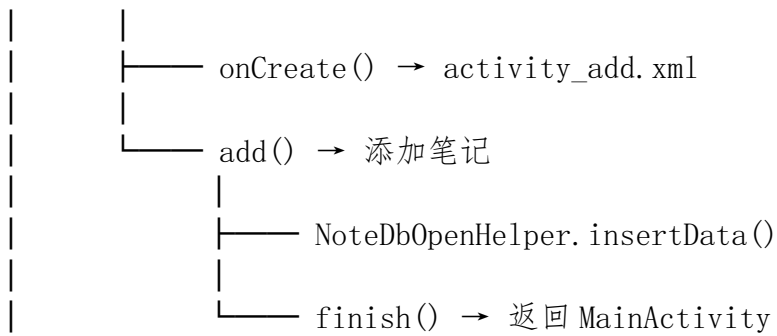
助类



List<Note>

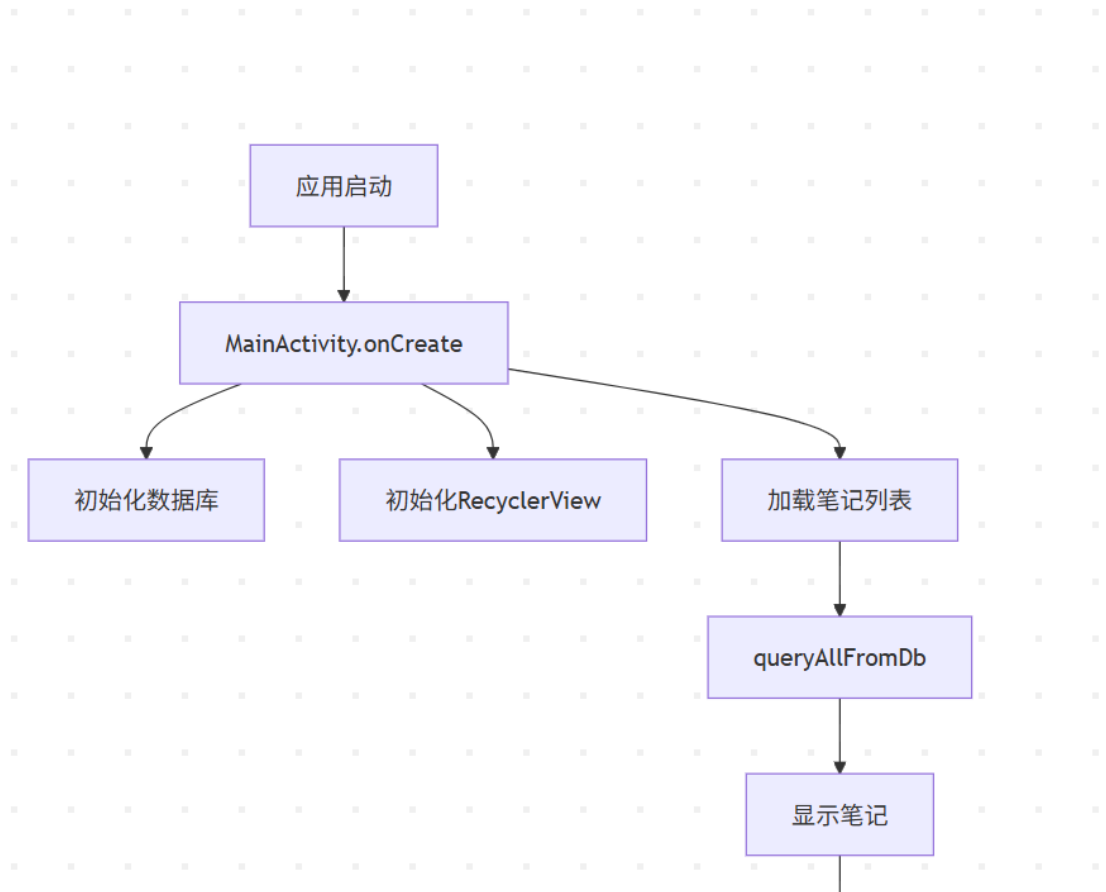


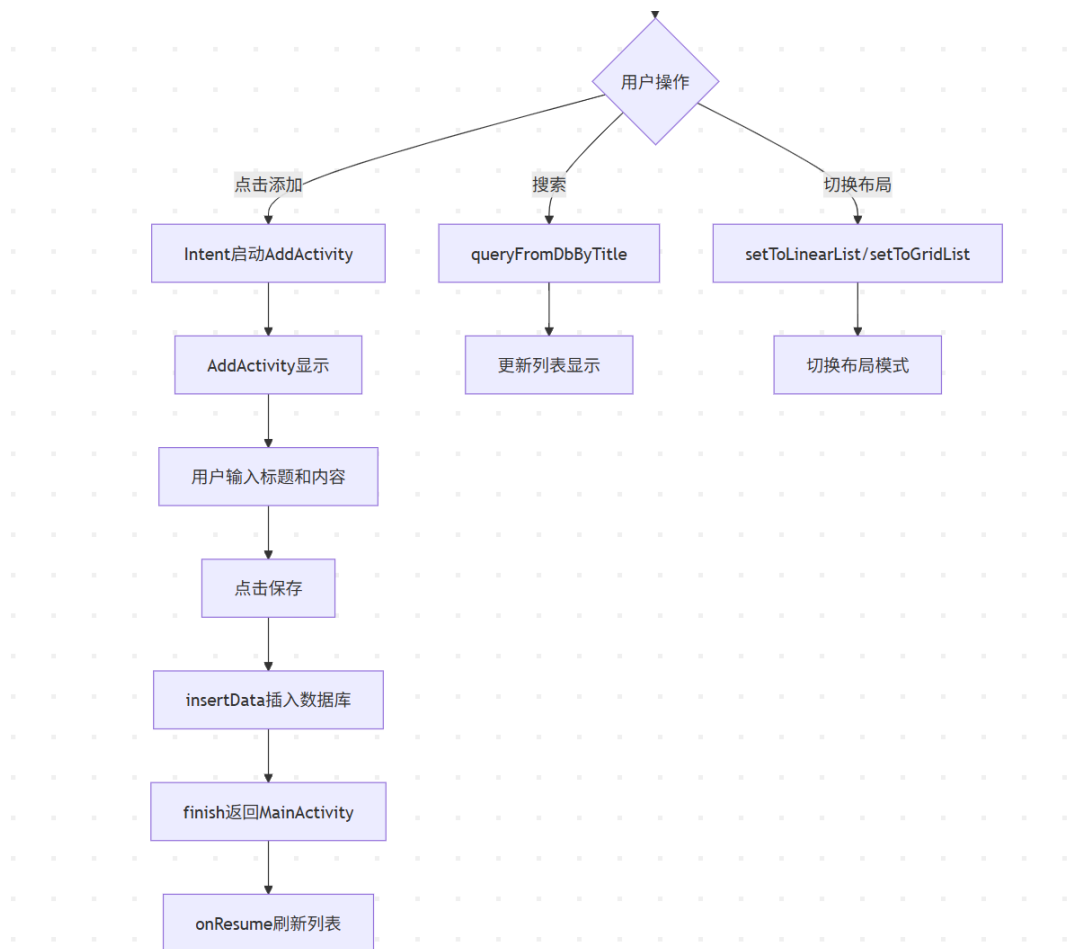
AddActivity





流程图：





2. 针对第二次 Android 实验的 4 个实验样例：

- a) LED 灯
- b) 蜂鸣器
- c) 温度采集
- d) 串口通信

针对不同 GPIO，进行各个案例的工程文件结构分析以及主活动文件

MainActivity.java 如何完成对于外设的驱动。

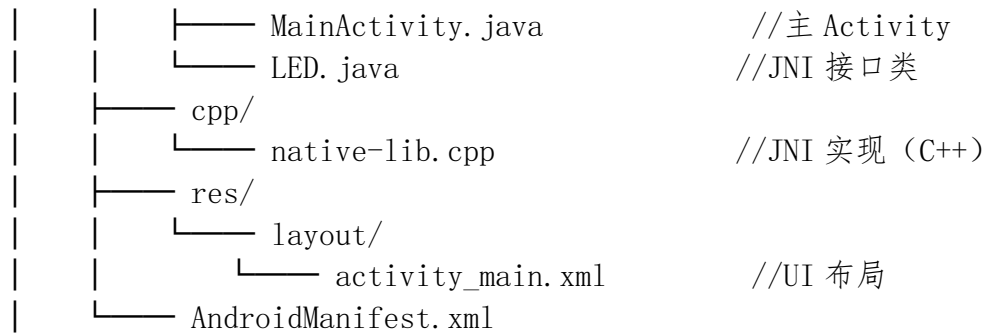
A:

工程文件结构：

LED/

|—— app/src/main/

| |—— java/com/farsight/led/



核心代码:

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    LED led = new LED();           // LED驱动类实例
    boolean IsLight_1_On = false;   // LED1状态标志
    boolean IsLight_2_On = false;   // LED2状态标志
    private ImageButton LedButton_1;
    private ImageButton LedButton_2;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        LedButton_1 = (ImageButton)findViewById(R.id.buttonOne);

        // LED1按钮点击事件
        LedButton_1.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (IsLight_1_On) {
                    // 关闭LED1

                    ((ImageButton)view).setBackground(getDrawable(R.drawable.pic_bulboff));
                    led.open();           // 打开设备文件
                    led.LedOff1();        // 调用JNI方法关闭LED1
                    led.close();          // 关闭设备文件
                } else {
                    // 打开LED1

                    ((ImageButton)view).setBackground(getDrawable(R.drawable.pic_bulbon));
                    led.open();
                    led.LedOn1();          // 调用JNI方法打开LED1
                    led.close();
                }
                IsLight_1_On = !IsLight_1_On; // 切换状态
            }
        });

        // LED2按钮类似处理
        // ...
    }
}
```

Activity 负责 UI 交互

通过 LED 类调用 JNI 方法控制硬件

每次操作都需要 open() → 控制 → close()

LED.java

```
public class LED {  
    static {  
        System.loadLibrary("led");           // 加载JNI库 (libled.so)  
    }  
  
    // JNI方法声明 (native关键字)  
    public native int open();                 // 打开设备文件  
    public native int close();                // 关闭设备文件  
    public native int LedOn1();               // 打开LED1  
    public native int LedOff1();              // 关闭LED1  
    public native int LedOn2();               // 打开LED2  
    public native int LedOff2();              // 关闭LED2  
}
```

JNI 机制:

static {}: 类加载时执行, 加载本地库

native: 声明本地方法, 实现在 C/C++ 中

库名 "led" 对应 libled.so 文件

native-lib.cpp (JNI 实现 - GPIO 驱动)

```

#include <jni.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <unistd.h>

// IO控制命令定义 (与Linux驱动中的定义一致)
#define LED1_ON      _IO('x',1)           // LED1打开命令
#define LED1_OFF     _IO('x',0)           // LED1关闭命令
#define LED2_ON      _IO('z',1)           // LED2打开命令
#define LED2_OFF     _IO('z',0)           // LED2关闭命令

int fd = 0;                               // 设备文件描述符

// JNI函数: 打开LED设备
extern "C" JNIEXPORT jint JNICALL
Java_com_farsight_led_LED_open(JNIEnv* env, jobject /* this */) {
    // 打开Linux设备文件 (字符设备驱动)
    fd = open("/dev/leds_ctl", O_RDWR);
    if (-1 == fd) {
        __android_log_print(ANDROID_LOG_INFO, "led", "open /dev/leds_ctl
Error");
    } else {
        __android_log_print(ANDROID_LOG_INFO, "led", "open /dev/leds_ctl
success");
    }
    return fd;
}

// JNI函数: 打开LED1
extern "C" JNIEXPORT jint JNICALL
Java_com_farsight_led_LED_LedOn1(JNIEnv* env, jobject /* this */) {
    // 使用ioctl系统调用控制设备
    ioctl(fd, LED1_ON);                     // 发送LED1_ON命令到驱动
    return 0;
}

// JNI函数: 关闭LED1
extern "C" JNIEXPORT jint JNICALL
Java_com_farsight_led_LED_LedOff1(JNIEnv* env, jobject /* this */) {
    ioctl(fd, LED1_OFF);                     // 发送LED1_OFF命令到驱动
    return 0;
}

```

GPIO 驱动关键点:

1. 设备文件操作:

/dev/leds_ctl: Linux 字符设备文件

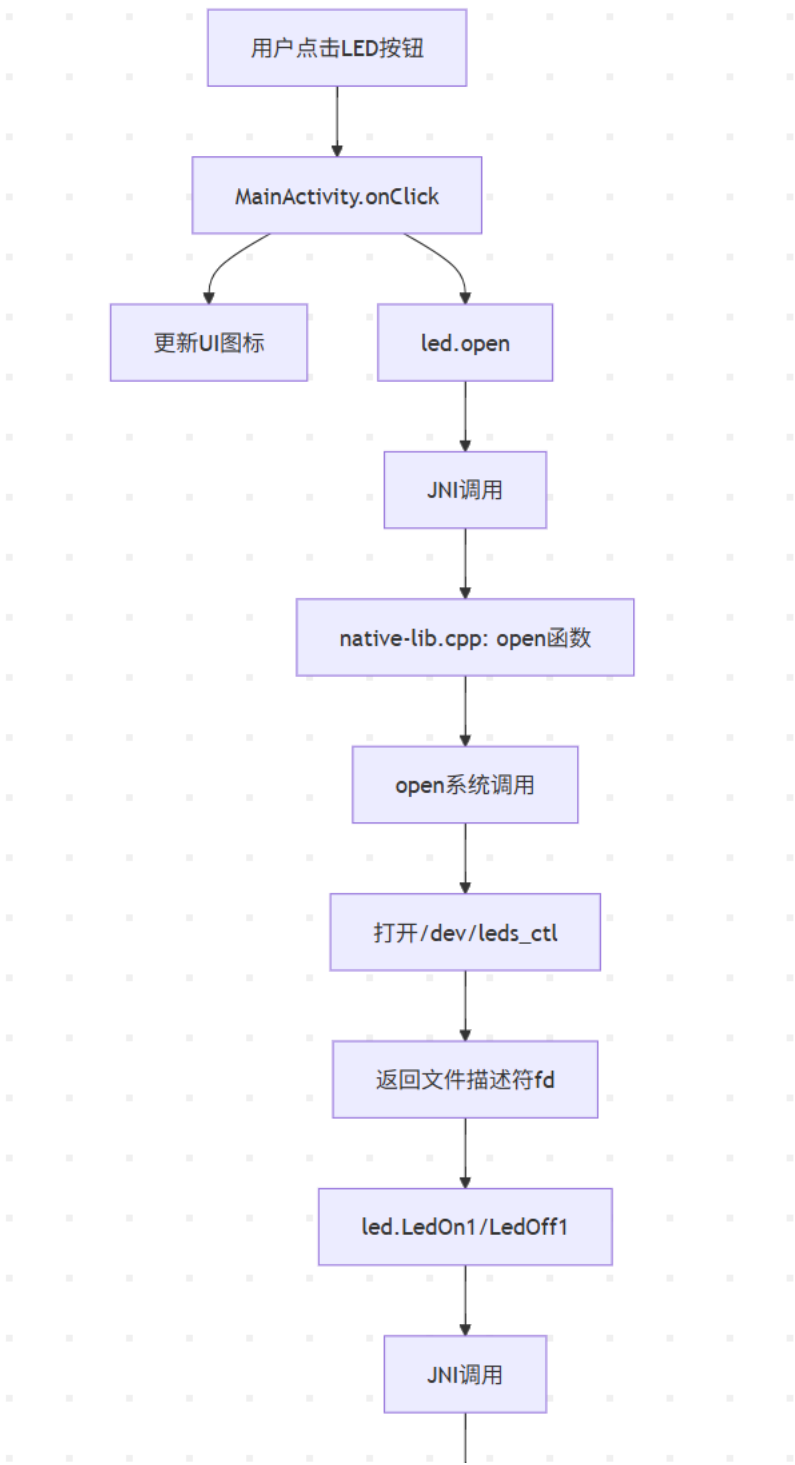
open(): 打开设备文件, 返回文件描述符

close(): 关闭设备文件

2. ioctl 系统调用:

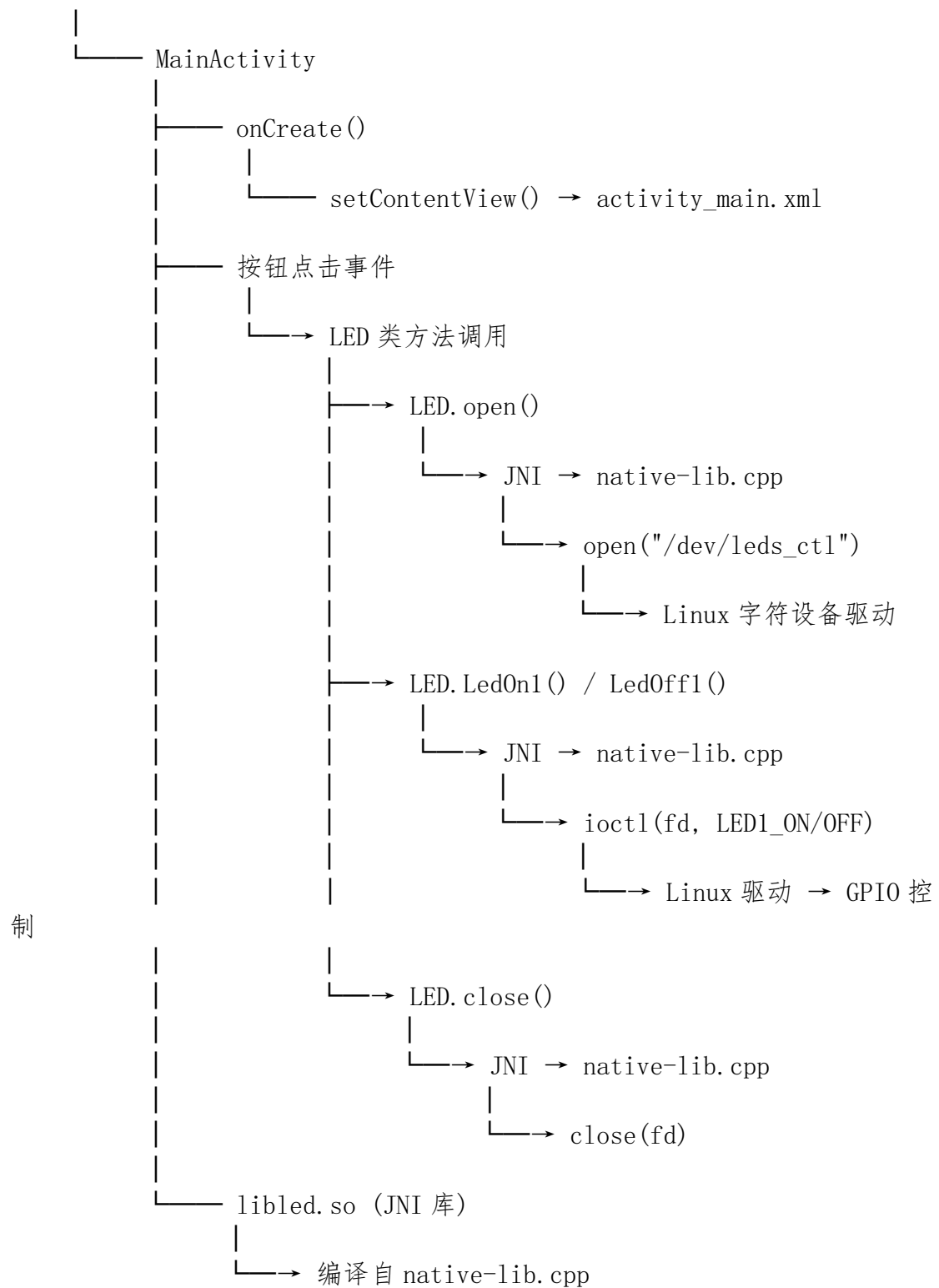
ioctl(fd, cmd): 向设备发送控制命令
LED1_ON/OFF: 自定义 IO 控制命令
驱动根据命令控制 GPIO 引脚

顺序图:



工程文件调用关系:

AndroidManifest.xml



B:
核心代码:

```

@Override
public void onClick(View view) {
    switch (view.getId()) {
        case R.id.button1:
            // 打开设备
            if(buzzer.open() == -1) {
                Toast.makeText(this, "设备打开失败!",
                    Toast.LENGTH_SHORT).show();
                return;
            }
            buzzer.BuzzerOn();           // 打开蜂鸣器
            buzzer.close();
            break;

        case R.id.button2:
            if(buzzer.open() == -1) {
                Toast.makeText(this, "设备打开失败!",
                    Toast.LENGTH_SHORT).show();
                return;
            }
            buzzer.BuzzerOff();          // 关闭蜂鸣器
            buzzer.close();
            break;
    }
}

```

```

public class Buzzer {
    static {
        System.loadLibrary("buzzer");    // 加载libbuzzer.so
    }
    public native int open();
    public native int close();
    public native int BuzzerOn();        // 打开蜂鸣器
    public native int BuzzerOff();       // 关闭蜂鸣器
}

```

通过 JNI 调用 C++代码控制 GPIO

C:

核心代码:

```

public class MainActivity extends AppCompatActivity {
    TextView val;                                // 温度值显示
    Button start_btn;
    Button close_btn;
    Boolean sensorflag = false;                  // 传感器状态标志
    double data = 0;
    temp tmp = new temp();                       // 温度传感器驱动类

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        val = findViewById(R.id.text);

        start_btn = findViewById(R.id.start_btn);
        start_btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (sensorflag == false) {
                    sensorflag = true;
                    tmp.open();                    // 打开温度传感器设备
                    new TimeThread().start();    // 启动数据采集线程
                }
            }
        });

        close_btn = findViewById(R.id.close_btn);
        close_btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (sensorflag == true) {
                    tmp.close();                  // 关闭设备
                    sensorflag = false;
                }
            }
        });
    }
}

```

```

// 数据采集线程：每隔200ms读取一次温度
public class TimeThread extends Thread {
    @Override
    public void run() {
        super.run();
        do {
            if (sensorflag == true) {
                try {
                    Thread.sleep(200); // 休眠200ms
                    Message msg = new Message();
                    msg.what = 1;
                    handler.sendMessage(msg); // 发送消息到主线程
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            } else {
                try {
                    Thread.sleep(200);
                    Message msg = new Message();
                    msg.what = 2;
                    handler.sendMessage(msg);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        } while (true);
    }
}

// Handler：在主线程中更新UI
private Handler handler = new Handler(new Handler.Callback() {
    @Override
    public boolean handleMessage(Message msg) {
        switch (msg.what) {
            case 1:
                data = tmp.read(); // 读取温度值 (JNI调用)
                val.setText(String.format("%.3f", data) + "°C");
                break;
            case 2:
                val.setText("传感器未打开");
                break;
        }
        return false;
    }
});

```

关键机制：

1. 多线程：

TimeThread：后台线程，定期读取传感器数据
避免阻塞主线程（UI 线程）

2. Handler 机制：

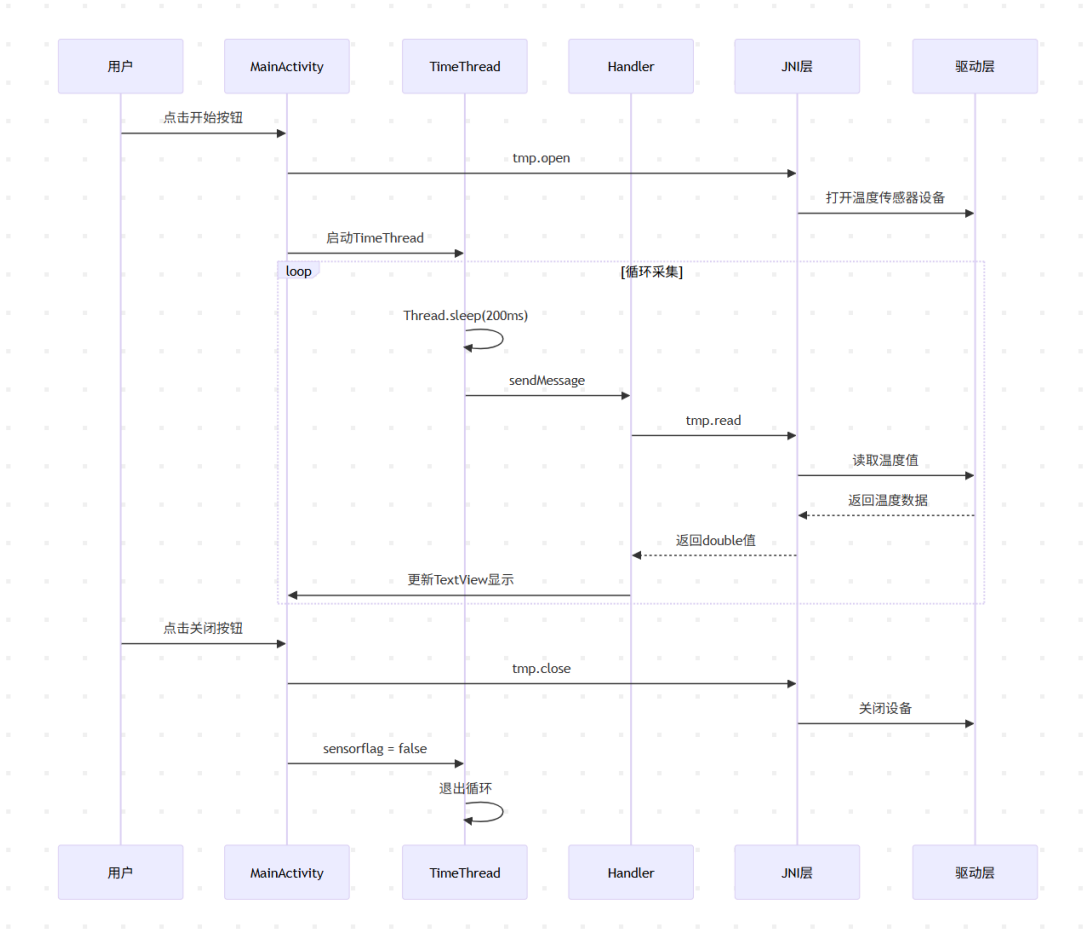
子线程不能直接更新 UI

通过 Handler 将数据传递到主线程更新 UI

3. 持续采集：

使用 do-while(true) 循环
每隔 200ms 读取一次温度

流程图：



D:

核心代码：

```

@Override
public void onClick(View v) {
    switch (v.getId()) {
        case R.id.send1:
            // 发送数据
            String m = ET1.getText().toString() + "\n";
            com.write(m.getBytes()); // 写入串口
            msglist.append("[发送]" + m);
            ET1.setText("");
            break;

        case R.id.open_serial:
            // 打开串口
            int ret = com.open(serial, Baudrate); // 传入串口和波特率
            if (ret > 0) {
                Toast.makeText(MainActivity.this, "串口打开成功",
Toast.LENGTH_LONG).show();
                OPENSERIAL.setEnabled(false);
                CLOSESERIAL.setEnabled(true);
                SEND.setEnabled(true);
                // 启动接收线程
                myThread = new MyThread();
                myThread.start();
            }
            break;

        case R.id.close_serial:
            // 关闭串口
            if (myThread != null) {
                com.close();
                myThread = null;
            }
            SEND.setEnabled(false);
            break;
    }
}
}

```

```

// 接收数据线程
class MyThread extends Thread {
    @Override
    public void run() {
        while (true) {
            byte[] bytes = com.read(); // 读取串口数据
            if(bytes == null) {
                break; // 读取失败，退出循环
            }
            String string = new String(bytes);
            String finalString = string;
            // 在主线程中更新UI
            runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    msglist.append("[接收]" + finalString + "\n");
                }
            });
        }
        // 串口关闭后的UI更新
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                OPENSERIAL.setEnabled(true);
                CLOSESERIAL.setEnabled(false);
            }
        });
    }
}

```

串口通信关键点：

1. 串口参数配置：

串口设备：/dev/ttyS0 或/dev/ttyS4

波特率：2400-115200

2. 双向通信：

write()：发送数据

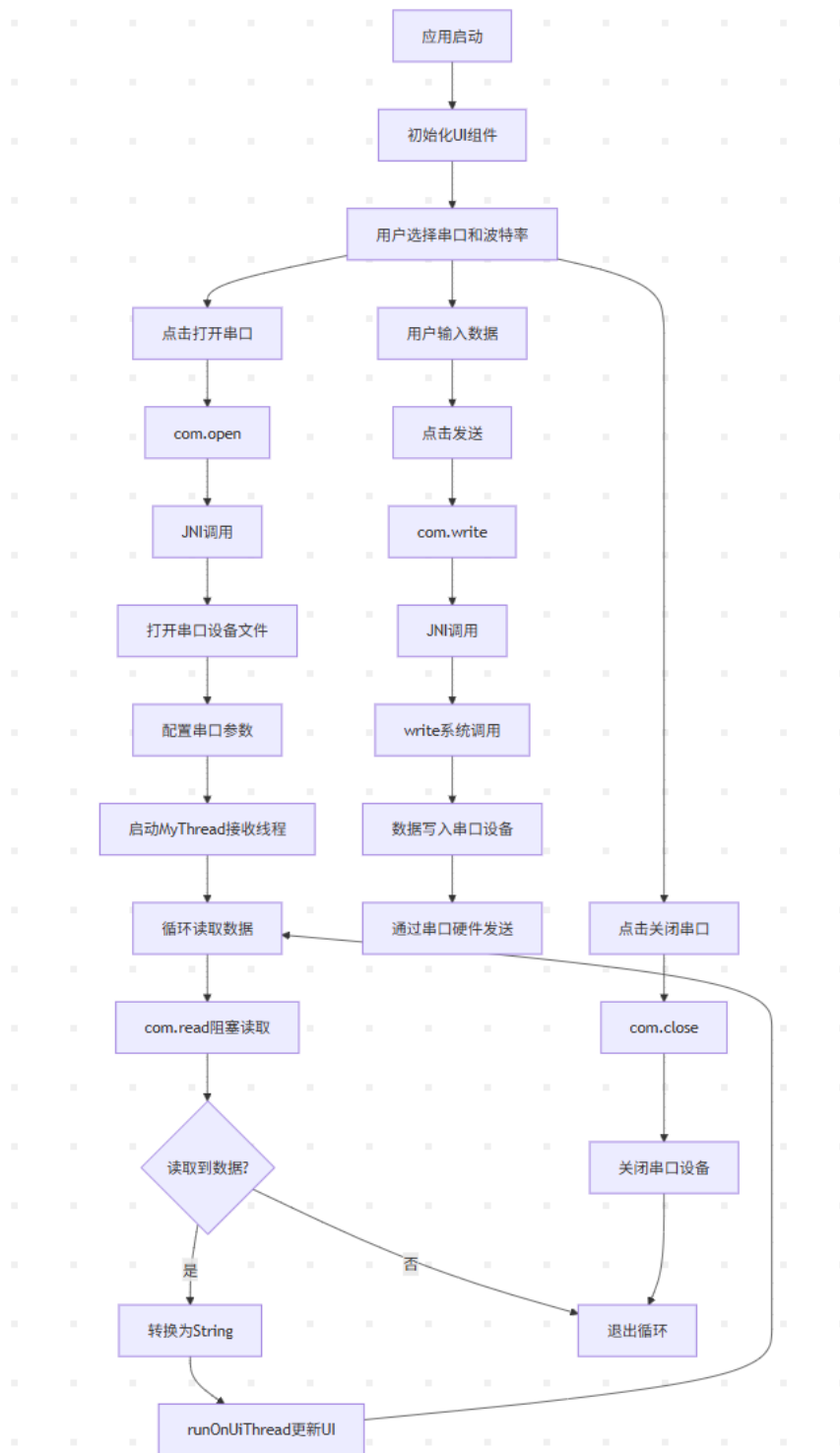
read()：接收数据（阻塞读取）

3. 多线程处理：

MyThread：后台线程持续读取串口数据

runOnUiThread()：在主线程中更新 UI

流程图



Android 与 Linux 设备驱动对比分析

对比项	Android 方式	Linux 原生方式
开发语言	Java + JNI + C++	C/C++
应用层	Android Activity	Linux 用户空间程序
接口层	JNI 桥接	直接系统调用

系统调用	通过 JNI 调用	直接调用
设备文件	/dev/leds_ctl	/dev/leds
控制方式	ioctl()	ioctl()
驱动层	Linux 字符设备驱动	Linux 字符设备驱动
GPIO 操作	驱动内部实现	驱动内部实现
代码复杂度	较高（多层）	较低（直接）
可移植性	Android 平台	Linux 平台
安全性	应用层隔离	需要权限管理

3. 代码剖析要求：

- a) 第一次 Android 实验：
 - i. 作图描述每个工程中各个文件之间的调用关系；
 - ii. 核心代码中对于不同模块（activity,intent,content provider, service）的实际应用剖析（代码注解+流程图）
- b) 第二次 Android 实验：
 - i. 工程文件中如何完成对于 GPIO 通用外部设备的驱动？（可采用流程图，代码加注等形式）
 - ii. 比较 Android 对于设备的控制与 linux 设备驱动案例的不同（从 LED 灯、蜂鸣器、温度采集中任选一个比对分析）。（可采用流程图，代码加注等形式）

4. 作业提交：

- a) 命名：姓名+学号+4.pdf
- b) 提交至数字化教学平台 course.xmu.edu.cn
- c) 截止时间：12 月 21 晚 12:00